

Build & Deploy – Homework 2

Luca Eterno – Rosario Grasso

1. Requisiti

Per prendere visione dei requisiti consultare il build_e_deploy dell'homework precedente.

2. Configurazione delle variabili d'ambiente

Prima di avviare il sistema è necessario predisporre un file `.env` nella root del progetto contenente le variabili sensibili utilizzate dai container.

Esempio di file `.env`:

```
MYSQL_ROOT_PASSWORD=strong_root_password
USER_DB_PASSWORD=usermgrpwd
DATA_DB_PASSWORD=userdtpwd
```

```
CLIENT_ID=opensky_client_id
CLIENT_SECRET=opensky_client_secret
```

```
JWT_secret = jwt_secret
```

*per generare il `JWT_secret` utilizzare il comando `openssl rand -hex 32`

3. Build dei container

Per costruire tutte le immagini Docker definite nel progetto è sufficiente eseguire:

```
docker compose build
```

4. Avvio dell'infrastruttura

L'avvio completo del sistema avviene tramite:

```
docker compose up
```

Oppure, per l'esecuzione in background:

```
docker compose up -d
```

Durante la fase di bootstrap:

- I database MySQL vengono inizializzati tramite script SQL montati come volume
- Kafka attende il broker attivo e crea automaticamente i topic richiesti
- I consumer Kafka restano in attesa finché broker e topic non risultano disponibili
- Il Gateway viene avviato solo dopo la disponibilità dei servizi backend

Grazie agli healthcheck definiti in `docker-compose`, l'ordine di avvio dei servizi è gestito in modo automatico.

5. Accesso ai servizi

Gateway (API REST)

Il Gateway Nginx rappresenta l'unico punto di accesso alle API REST:

- **HTTP:**
`http://progettoeternograsso.com:8080`
- **HTTPS (self-signed):**
`https://progettoeternograsso.com:8443`

Le API sono raggiungibili tramite:

- `/api/users/...` → User Manager
- `/api/data/...` → Data Collector

MailHog

L'interfaccia web di MailHog è disponibile all'indirizzo:

`http://localhost:8025`

Qui è possibile verificare la ricezione delle email di notifica generate dal sistema.

7. Creazione automatica dei topic Kafka

I topic Kafka necessari al funzionamento del sistema vengono creati automaticamente dal container `kafka-setup`:

- `to-alert-system`
- `to-notifier`

Il container viene eseguito una sola volta all'avvio e termina dopo aver completato la creazione. In caso di problemi, è possibile verificarne l'esito consultando i log con:

```
docker logs kafka-setup
```

8. Testing del sistema

Utilizzare `DSBD-HW2.postman_collection.json` da importare su POSTMAN

9. Arresto del sistema

Per arrestare tutti i container mantenendo i volumi persistenti:

```
docker compose down
```

Per rimuovere anche i volumi (reset completo dei dati):

```
docker compose down -v
```